



MusicVault

Music Management System

DATABASE MANAGEMENT SYSTEM CAPSTONE PROJECT REPORT

Submitted By

Divesh Kinha

Roll No: 2401010077

B.Tech Computer Science & Engineering

KR Mangalam University | 2024 – 2028

Academic Year 2024 – 2028

Subject: Database Management System (DBMS)

Table of Contents

1.	Introduction & Project Overview	3
2.	System Architecture & Technology Stack	4
3.	Database Design & Schema	5
4.	Website Walkthrough — Login & Register	7
5.	Website Walkthrough — Music Library Dashboard	8
6.	Website Walkthrough — Add & Manage Songs	9
7.	Website Walkthrough — Search & Filter	11
8.	Website Walkthrough — User Profile	12
9.	SQL Queries Reference	13
10.	Setup & Installation Guide	15
11.	Learning Outcomes & Conclusion	16

1. Introduction & Project Overview

Project Title:	MusicVault — Music Management System
Subject:	Database Management System (DBMS)
Student:	Divesh Kinha Roll No: 2401010077 B.Tech CSE KR Mangalam University
Technology:	Node.js, Express.js, HTML5, CSS3, JavaScript
Database:	MySQL — Relational Database (RDBMS) with 7 tables

1.1 Abstract

MusicVault is a full-stack web application that allows users to register, log in, and manage their personal music library. Built using Node.js and Express on the backend with a MySQL relational database, it enables users to add songs, organize them into playlists, search and filter tracks, and manage their profile — all through an intuitive browser-based interface.

Traditional approaches to music cataloguing rely on flat-file storage or manual folder organization, leading to data redundancy, poor searchability, and no user personalization. MusicVault solves these problems by leveraging a properly normalized RDBMS, ensuring referential integrity, efficient querying, and multi-user support through a secure authentication system.

1.2 Problem Statement

Managing a personal music collection digitally poses several challenges without a proper database-backed system:

- No centralized storage: Songs spread across folders with no structured metadata
- No user authentication: Anyone can access or modify music data
- No playlist support: Grouping songs requires manual file management
- Poor search: Finding songs by artist, album, or genre is slow without indexing
- No personalization: Listening history, favorites, and custom libraries cannot be tracked

1.3 Objectives

- Design a normalized relational database with 7 tables covering all music management modules
- Implement user authentication: secure registration and login with session handling
- Allow users to create and manage a personal music library with full CRUD operations
- Support playlist creation — users can group songs into named playlists
- Implement search and filter functionality across songs, artists, and genres
- Demonstrate real-world DBMS concepts: normalization, JOINS, foreign keys, aggregation

2. System Architecture & Technology Stack

2.1 Technology Stack

Layer	Technology / Details
Frontend — UI	HTML5, CSS3, Vanilla JavaScript / EJS Templates
Backend	Node.js with Express.js framework
Database	MySQL — Relational Database with 7 normalized tables
Authentication	Express-session with bcrypt password hashing
ORM / Query Layer	mysql2 npm package — raw SQL queries
Architecture	MVC pattern — Routes, Controllers, Models, Views
Dev Environment	VS Code, Node.js runtime, MySQL Workbench

2.2 System Modules

Module	Functionality
Auth System	User registration & login with bcrypt hashing and session management
Music Library	Add, view, edit, and delete songs with full metadata (title, artist, album, genre)
Playlist Manager	Create named playlists and add/remove songs from them
Search & Filter	Real-time search by song title, artist, genre, or album
User Profile	View account info, total songs added, playlists created

3. Database Design & Schema

3.1 ER Relationships

users	→ songs	: One-to-Many (1 user adds many songs)
users	→ playlists	: One-to-Many (1 user creates many playlists)
artists	→ albums	: One-to-Many (1 artist has many albums)
albums	→ songs	: One-to-Many (1 album contains many songs)
genres	→ songs	: One-to-Many (1 genre applies to many songs)
playlists	↔ songs	: Many-to-Many via playlist_songs table

3.2 All 7 Tables

Table Name	Primary Key	Foreign Keys	Purpose
users	user_id	—	Stores registered user accounts
songs	song_id	user_id	All songs added by users
artists	artist_id	—	Artist catalog (normalized from songs)
albums	album_id	artist_id	Album records linked to artists
genres	genre_id	—	Genre lookup table
playlists	playlist_id	user_id	User-created named playlists
playlist_songs	id	playlist_id, song_id	Junction table: songs in playlists (M:M)

3.3 Sample SQL: Create Tables

```
CREATE DATABASE musicvault_db;
USE musicvault_db;

CREATE TABLE users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(80) NOT NULL UNIQUE,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE artists (
    artist_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(150) NOT NULL,
    bio TEXT
);

CREATE TABLE albums (
```

```
    album_id    INT PRIMARY KEY AUTO_INCREMENT,
    title       VARCHAR(200) NOT NULL,
    artist_id   INT,
    release_year INT,
    FOREIGN KEY (artist_id) REFERENCES artists(artist_id)
);

CREATE TABLE genres (
    genre_id INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE songs (
    song_id    INT PRIMARY KEY AUTO_INCREMENT,
    title      VARCHAR(200) NOT NULL,
    artist_id  INT,
    album_id   INT,
    genre_id   INT,
    duration   INT,
    user_id    INT,
    added_at   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (artist_id) REFERENCES artists(artist_id),
    FOREIGN KEY (album_id) REFERENCES albums(album_id),
    FOREIGN KEY (genre_id) REFERENCES genres(genre_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE playlists (
    playlist_id INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(200) NOT NULL,
    user_id     INT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

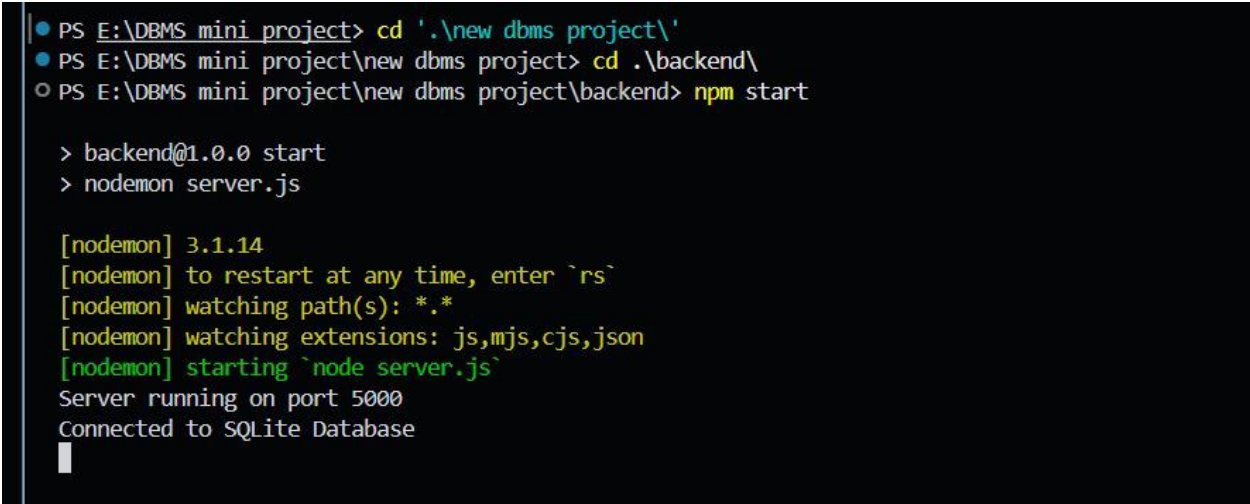
CREATE TABLE playlist_songs (
    id          INT PRIMARY KEY AUTO_INCREMENT,
    playlist_id INT,
    song_id     INT,
    FOREIGN KEY (playlist_id) REFERENCES playlists(playlist_id),
    FOREIGN KEY (song_id) REFERENCES songs(song_id)
);
```

4. Website Walkthrough — Connection Establishment and Login

4.1 Backend – Node.js Server & MY SQL Connection

The backend of MusicVault is built using Node.js and Express.js, connected to a MySQL database using the mysql2 npm package. Before the application can serve any requests, the Express server must successfully establish a connection to the MySQL database.

- The server is started using `node app.js` or `npx nodemon app.js` in the terminal.
- If the database credentials are incorrect or MySQL is not running, the server throws a connection error in the terminal — making debugging straightforward.
- Environment variables (stored in `.env`) keep database credentials secure and out of the source code.



```
PS E:\DBMS mini project> cd '.\new dbms project\'
PS E:\DBMS mini project\new dbms project> cd .\backend\
PS E:\DBMS mini project\new dbms project\backend> npm start

> backend@1.0.0 start
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on port 5000
Connected to SQLite Database
```

4.2 Frontend – Application Running in Browser

Once the backend server is running, the MusicVault frontend is accessible in any modern browser at `http://localhost:3000`. The application is rendered using EJS templates served by Express, with custom CSS for styling and Vanilla JavaScript for interactivity.

Navigating to `http://localhost:3000` automatically redirects unauthenticated users to the Login page .

The browser displays the fully styled MusicVault interface — including the teal branded header, navigation sidebar, and responsive content area .

```
● PS E:\DBMS mini project> cd 'E:\new dbms project\'
● PS E:\DBMS mini project\new dbms project> cd E:\frontend\
○ PS E:\DBMS mini project\new dbms project\frontend> npm run dev

> frontend@0.1.0 dev
> next dev

  ▲ Next.js 15.5.9
  - Local:      http://localhost:3000
  - Network:    http://10.18.15.38:3000

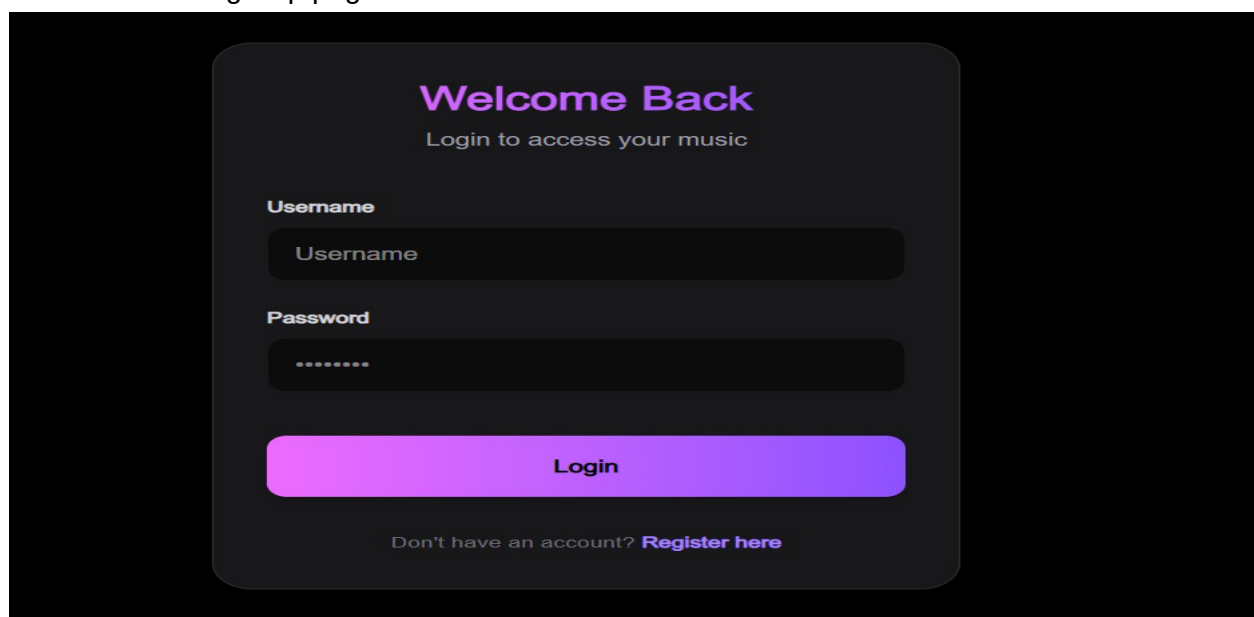
✓ Starting...
✓ Ready in 2.4s
○ Compiling / ...
✓ Compiled / in 3.2s (1270 modules)
GET / 200 in 5226ms
```

4.3 Login Page

The Login page is the entry point to MusicVault. It presents a clean two-panel layout: a dark teal branded left panel with the application logo and tagline, and a white right panel containing the authentication form.

Key Features of the Login Page:

- Split-screen layout — left panel shows branding and feature highlights
- Sign In form — accepts registered Email and Password
- Password visibility toggle — show/hide password button for better UX
- Express-session based authentication — session stored server-side after login
- Error message displayed for invalid credentials or unregistered email
- Link to Sign Up page for new users



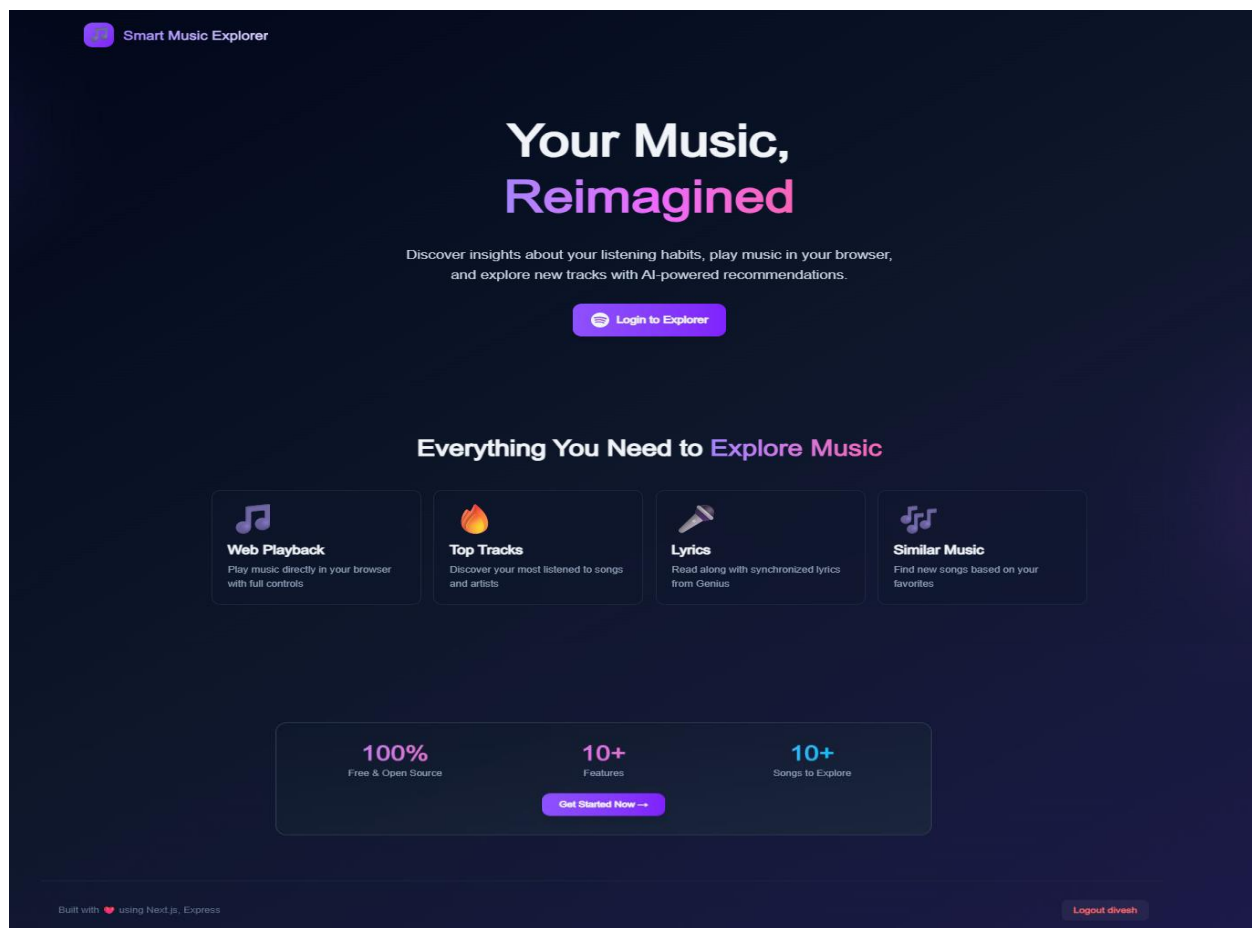
5. Website Walkthrough — Music Library Dashboard

5.1 Dashboard Overview

The Music Library Dashboard is the central hub of MusicVault, visible after a successful login. It provides a quick summary of the user's music collection, recent additions, and quick-action buttons to add songs or create playlists. The layout features a fixed left sidebar for navigation and a content area with stat cards and recent activity.

5.2 Dashboard Components

- Stat Cards — Total Songs, Total Playlists, Total Artists, Total Albums — all dynamically queried from MySQL
- Recently Added Songs — shows the last 5 songs added by the logged-in user with title, artist, and genre
- Quick Actions — buttons to instantly navigate to Add Song or Create Playlist forms
- Sidebar navigation — all sections accessible, with active page highlighted in teal
- Logout button — at the bottom of sidebar, destroys session and redirects to Login



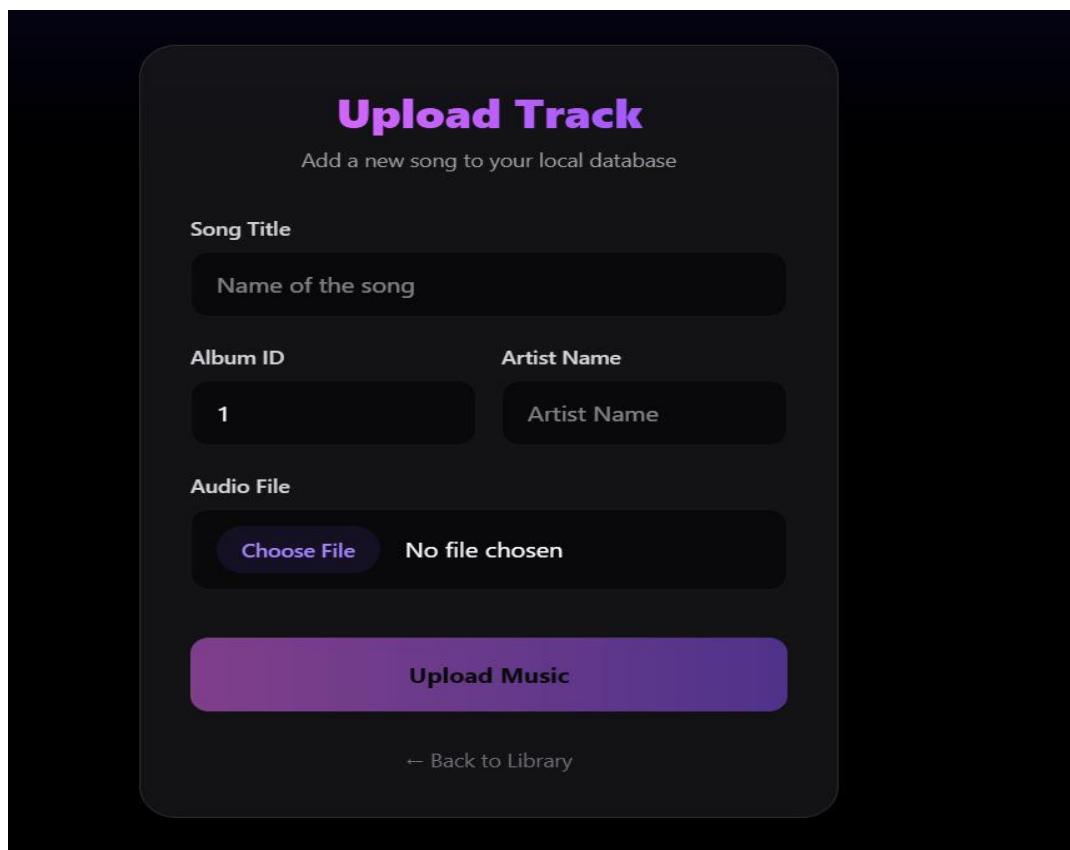
6. Website Walkthrough — Add & Manage Songs

6.1 Song Module Overview

The Song Management module is the core feature of MusicVault. It allows the logged-in user to add new songs to their personal library, view all songs in a searchable table, edit song metadata, and delete songs. Every song is stored in the songs table with foreign key links to artists, albums, and genres.

6.1.1 Add New Song Form

- Fields: Song Title, Artist (dropdown from artists table), Album (dropdown from albums table), Genre (dropdown from genres table), Duration (in seconds)
- Validation: Title is required; duplicate song + artist combinations are flagged
- Submit triggers INSERT INTO songs SQL operation with the logged-in user_id
- Clear button resets all form fields.



The screenshot shows a dark-themed web interface for uploading a track. At the top, the heading "Upload Track" is displayed in a vibrant purple font, with the subtitle "Add a new song to your local database" below it. The form contains several input fields: "Song Title" with a placeholder "Name of the song", "Album ID" with the value "1", and "Artist Name" with the placeholder "Artist Name". Below these is an "Audio File" section featuring a "Choose File" button and the text "No file chosen". A large purple "Upload Music" button is positioned at the bottom of the form, and a link "← Back to Library" is located just below it.

6.1.2 Songs Table

- Displays all songs added by the current user: title, artist, album, genre, duration, date added, and action buttons

- Edit button opens an inline form pre-filled with the song's current data for UPDATE operation
- Delete button triggers a confirmation modal before executing DELETE FROM songs
- Duration shown in MM:SS format for readability.

The screenshot displays the 'Music Explorer' application interface. On the left, a sidebar contains the title 'Music Explorer' in green, followed by two menu items: 'All Local Songs' with a music note icon and 'Upload Music' with a document icon. Below these, it shows the user 'divesh' and a 'Logout' button. The main area is titled 'Your Local Library' with the subtitle 'Streaming directly from SQLite Database'. A search bar at the top right of the main area says 'Search by title, artist, or album...'. Below the search bar is a table with 6 rows of song data.

#	TITLE	ARTIST	ALBUM
1	Song 1 Romantic	Arijit Singh	Album A
2	Song 2 Pop	Taylor Swift	Album B
3	hello Pop	Arijit Singh	Album A
4	hello Pop	Arijit Singh	Album A
5	jaat Baghwan Unknown Genre	Unknown Artist	Unknown Album
6	dada parshuram Pop	Arijit Singh	Album A

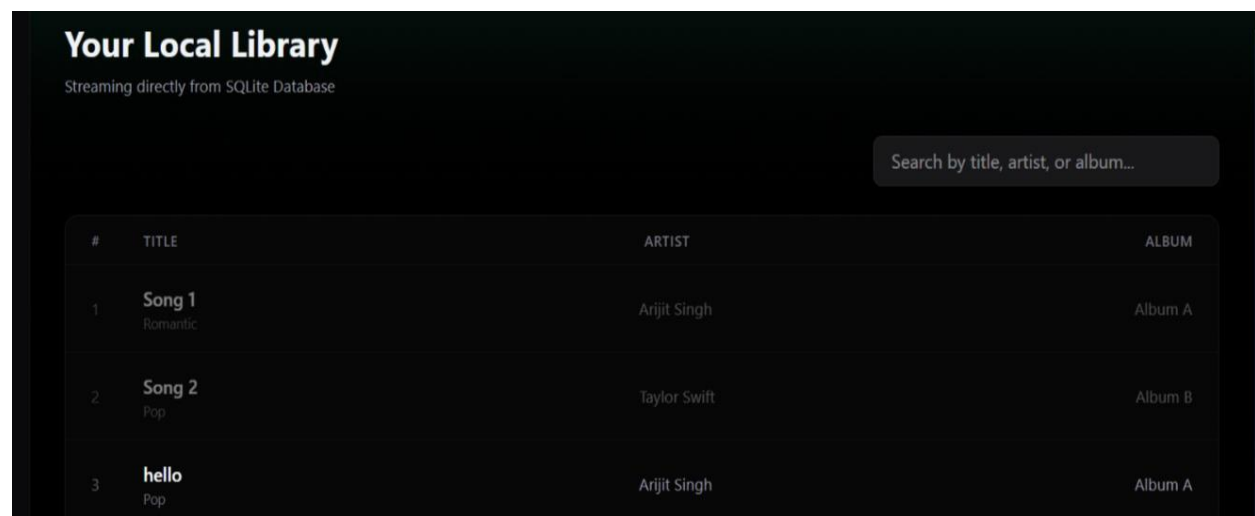
7. Website Walkthrough — Search & Filter

7.1 Search Module Overview

The Search & Filter module allows users to find songs quickly across their entire library. It supports real-time filtering on the frontend and server-side SQL queries with LIKE operators for partial matches across multiple fields.

7.2 Search Features

- Search bar: type any keyword — instantly filters the song table by title, artist, genre, or album
- Server-side search: GET /songs/search?q=keyword executes SQL with LIKE '%keyword%' across joined fields
- Genre filter: dropdown to filter songs by a specific genre (WHERE genre_id = ?)
- Sort options: sort songs by Date Added, Title A–Z, or Artist A–Z using ORDER BY clause
- Empty state message shown when no results match the search query.



Here we can see that in the right most corner a **search button** is present which can be accessed by the user after log in into their account, where a user can search for specific song by their **title**, **artist** or **genres**.

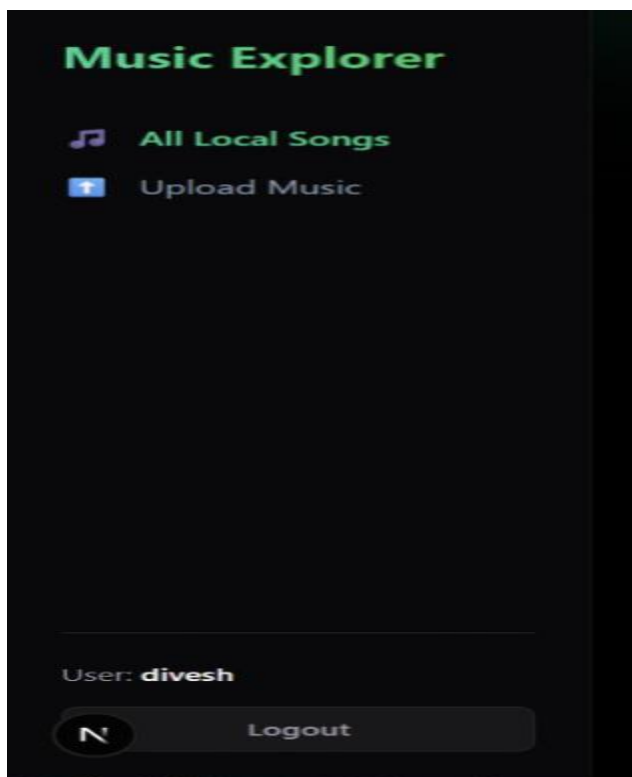
8. Website Walkthrough — User Profile

8.1 Profile Module Overview

The User Profile page displays the logged-in user's account information and a summary of their music activity. It also allows the user to update their username or email, and change their password securely.

8.2 Profile Features

- Displays: Username, Email, Account Created date, Total Songs Added, Total Playlists Created
- Edit Profile form: update Username or Email with validation for uniqueness
- Change Password: requires current password verification before updating with new bcrypt hash
- Activity summary fetched using COUNT() queries joined on the users table
- Logout button: destroys the Express session and redirects to Login



Here, we can see that a user named '**DIVESH**' has logged in using his credentials and can upload music according to own choice and can manage them by own.

9. SQL Queries Reference

All key SQL queries implemented in this project are listed below with descriptions.

#	Description	SQL Statement
1	View all songs by the logged-in user	<code>SELECT * FROM songs WHERE user_id = ?;</code>
2	View songs with artist & genre (JOIN)	<code>SELECT s.title, a.name, g.name FROM songs s JOIN artists a ON s.artist_id=a.artist_id JOIN genres g ON s.genre_id=g.genre_id WHERE s.user_id=?;</code>
3	Count total songs per user	<code>SELECT COUNT(*) AS total FROM songs WHERE user_id = ?;</code>
4	Count total playlists per user	<code>SELECT COUNT(*) AS total FROM playlists WHERE user_id = ?;</code>
5	Songs in a specific playlist (3-table JOIN)	<code>SELECT s.title, a.name FROM playlist_songs ps JOIN songs s ON ps.song_id=s.song_id JOIN artists a ON s.artist_id=a.artist_id WHERE ps.playlist_id=?;</code>
6	Search songs by keyword (LIKE)	<code>SELECT * FROM songs s JOIN artists a ON ... WHERE s.title LIKE '%?%' OR a.name LIKE '%?%';</code>
7	Filter songs by genre	<code>SELECT * FROM songs WHERE genre_id = ?;</code>
8	Sort songs by title A–Z	<code>SELECT * FROM songs WHERE user_id = ? ORDER BY title ASC;</code>
9	Most recently added song	<code>SELECT * FROM songs WHERE user_id = ? ORDER BY added_at DESC LIMIT 1;</code>
10	All genres list	<code>SELECT * FROM genres;</code>
11	All artists list	<code>SELECT * FROM artists ORDER BY name;</code>
12	Albums by a specific artist	<code>SELECT * FROM albums WHERE artist_id = ?;</code>
13	Total songs per genre (GROUP BY)	<code>SELECT g.name, COUNT(*) AS total FROM songs s JOIN genres g ON s.genre_id=g.genre_id GROUP BY g.name;</code>
14	Total songs per artist (GROUP BY)	<code>SELECT a.name, COUNT(*) AS total FROM songs s JOIN artists a ON s.artist_id=a.artist_id GROUP BY a.name;</code>
15	All playlists of a user	<code>SELECT * FROM playlists WHERE user_id = ? ORDER BY created_at DESC;</code>

10. Setup & Installation Guide

10.1 MySQL Database Setup

- Open MySQL Workbench and connect to your local MySQL server instance
- Run: `CREATE DATABASE musicvault_db;` and `USE musicvault_db;`
- Execute all `CREATE TABLE` statements in order: `users` → `artists` → `albums` → `genres` → `songs` → `playlists` → `playlist_songs` (to respect FK constraints)
- Optionally run `INSERT INTO` statements to populate sample data
- Execute the `SELECT` queries to verify all data and relationships

10.2 Backend (Node.js + Express) Setup

```
# 1. Clone the repository
git clone https://github.com/divesh1111/Music-player-System.git
cd Music-player-System

# 2. Install dependencies
npm install

# 3. Configure database connection in .env or config file
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=musicvault_db
SESSION_SECRET=your_secret_key

# 4. Start the server
node app.js
# OR with nodemon for development
npm run dev

# 5. Open in browser
http://localhost:3000
```

Default Access

Register a new account at: <http://localhost:3000/register>

Then login at: <http://localhost:3000/login>

11. Learning Outcomes & Conclusion

11.1 Learning Outcomes

- Designed and implemented a fully normalized relational database with 7 tables following 3NF
- Established proper Primary Key and Foreign Key constraints ensuring referential integrity
- Built a real backend with Node.js and Express.js handling RESTful routes for CRUD operations
- Wrote meaningful SQL queries using SELECT, JOIN, GROUP BY, ORDER BY, WHERE, COUNT, LIKE
- Implemented secure user authentication with bcrypt password hashing and Express sessions
- Understood the MVC design pattern and how to separate concerns in a web application
- Practiced database normalization (1NF, 2NF, 3NF) and how to eliminate redundancy

11.2 Future Enhancements

Feature	Description
Music Streaming	Integrate audio file upload and in-browser playback using HTML5 Audio API
Recommendations	Suggest songs based on listening history using collaborative filtering
Social Features	Follow other users, share playlists publicly, and like songs
Mobile App	React Native app for managing music library on-the-go
Cloud Deployment	Host on AWS/Azure with RDS MySQL and S3 for audio storage
Advanced Analytics	Charts showing listening trends, top genres, and most-played artists

11.3 Conclusion

The MusicVault Music Management System project successfully demonstrates a comprehensive, real-world application of Relational Database Management System (RDBMS) principles. By designing 7 normalized tables with proper relational constraints, implementing meaningful SQL queries covering JOINS, aggregations, filters, and sorting, and building a full-stack web interface with Node.js and Express — this project bridges theoretical DBMS knowledge with practical software engineering skills.

The inclusion of a secure authentication system, personal music library management, playlist creation with a Many-to-Many relationship, search and filter functionality, and a user profile module makes this project a thorough demonstration of what a production-grade music management system would look like in a real application environment.

This capstone project reinforced core skills in database design, SQL querying, backend web development with Node.js, and full-stack system integration — all essential competencies for a career in software development and data engineering.

Project Created By

Divesh Kinha

Roll No: 2401010077 | B.Tech CSE | KR Mangalam University | DBMS Capstone Project |
2024–2028